ПМ3 Разработка модулей ПО.

РО 3.1 Понимать и применять принципы объектно-ориентированного и асинхронного программирования.

Тема 1. Введение в ООП.

Лекция 4. Прототипы и прототипное наследование.

Цель занятия:

Познакомиться с прототипами в JavaScript и понять механизм наследования через цепочку прототипов.

Учебные вопросы:

1. Основы прототипов.

2. Цепочка прототипов. Механизм поиска свойств.

1. Основы прототипов.

Прототип в JavaScript — это механизм, который позволяет объектам наследовать свойства и методы друг от друга.

Каждый объект в JavaScript имеет внутреннюю ссылку на другой объект, который называется его прототипом

Когда вы пытаетесь получить доступ к свойству объекта, JavaScript сначала ищет это свойство в самом объекте, а затем — в его прототипе, и так далее, пока не найдет его или не дойдет до конца цепочки ссылок (до **null**).

Свойство prototype у функций-конструкторов.

Функции-конструкторы в JavaScript — это функции, используемые для создания объектов.

Когда вы создаете объект с помощью функции-конструктора (с использованием ключевого слова new), JavaScript устанавливает свойство prototype этой функции в качестве прототипа создаваемого объекта.

Пример:

```
function Person(name) {
    this.name = name;
}

console.log(typeof Person.prototype) // object

// Добавляем метод greet в прототип Person
Person.prototype.greet = function() {
    console.log(`Hello, my name is ${this.name}`);
};

const alice = new Person('Alice');
alice.greet(); // "Hello, my name is Alice"
```

Как это работает:

- Создание объекта: Когда вы создаете объект alice с помощью new Person('Alice'), JavaScript создает новый объект и устанавливает его внутреннее свойство [[Prototype]] на Person.prototype.
- Наследование: Когда вы вызываете alice.greet(), JavaScript ищет метод greet сначала в объекте alice, а затем в Person.prototype, где он и находит его.

Зачем это нужно?

Использование прототипов позволяет экономить память, так как все экземпляры одного и того же типа могут использовать одни и те же методы, определенные в прототипе, вместо того чтобы иметь свои собственные копии этих методов.

Таким образом, прототипы в JavaScript являются ключевым элементом наследования и организации кода.

Различия между prototype, proto и [[Prototype]]

В JavaScript три термина — prototype, __proto__ и [[Prototype]] — относятся к механизму прототипного наследования, но они имеют разные значения и назначения.

[[Prototype]]:

- Внутреннее (скрытое) свойство любого объекта, которое хранит ссылку на прототип объекта, не доступное напрямую.
- Указывает на другой объект, из которого берутся свойства и методы, если их нет у самого объекта.
- По сути – ссылка на родителя.
- Доступ к нему получить напрямую нельзя.

Пример:

У вас есть объект **user,** если у него нет метода **getInfo()** JS пойдет искать его в **[[Prototype]].**

__proto__

- Старый, неформальный способ получить или задать [[Prototype]].
- Работает почти везде, но считается устаревшим.
- По сути это геттер/сеттер для [[Prototype]].

prototype

- Свойство функции-конструктора.
- Когда вы создаете объект с помощью **new**, именно **prototype** этой функции становится **[[Prototype]]**

```
function Person(name){
    this.name = name;
}
Person.prototype.sayHello = function(){
    console.log(`Hello, I'm ${name}`);
}

const alex = new Person("Alex");
alex.sayHello(); // Hello, I'm
```

- Здесь Alex.[[prototype]] === Person.prototype

Простыми словами:

- **prototype** – это чертеж для всех объектов, созданных через new.
- **[[Prototype]]** – это невидимая цепочка наследования для конкретного объекта.
- **__proto__** - это дверца, через которую можно заглянуть или поменять этот родитель (устаревший способ)

2. Цепочка прототипов. Механизм поиска свойств.

Как работает цепочка прототипов?

Объект: Когда вы создаете объект, он имеет ссылку на свой прототип, который может быть другим объектом или null.

Поиск свойства: Когда вы обращаетесь к свойству объекта:

- JavaScript сначала ищет это свойство в самом объекте.
- Если оно не найдено, JavaScript переходит к прототипу объекта (через свойство [[Prototype]] или __proto__).
- Этот процесс продолжается, пока не будет найдено свойство или не будет достигнут null.

Пример:

```
Object.prototype.sayHello = function(){console.log("Hello!")}

const animal = {
  makeSound: function(){
    console.log("Sound!")
  }
}

const dog = Object.create(animal);
dog.sayHello();
```

```
> dog

<  ▾ {} i
    ▾ [[Prototype]]: Object
      ▸ makeSound: ƒ ()
      ▾ [[Prototype]]: Object
        ▸ sayHello: ƒ ()
```

Механизм поиска свойства.

- Запрос свойства: Вы вызываете dog.sayHello().
- Поиск в объекте: JavaScript ищет метод sayHello в объекте dog. Он его не находит.
- Поиск в прототипе: JavaScript переходит к dog.prototype (animal)и ненаходит его.
- Поиск в прототипе прототипа: JavaScript переходит к animal.prototype (Object) и находит его там.
- Выполнение метода: Метод вызывается.

Базовый пример конструктора + prototype:

```
function Animal(name) {
  this.name = name; // собственное свойство
}

// метод в прототипе (общий для всех)
Animal.prototype.sayHi = function() {
  console.log("Hi, I'm " + this.name);
};

const cat = new Animal("Barsik");
const dog = new Animal("Sharik");

// name — у каждого объекта свой
cat.sayHi(); // Hi, I'm Barsik
dog.sayHi(); // Hi, I'm Sharik

// метод sayHi общий для всех объектов, лежит в Animal.prototype
console.log(cat.sayHi === dog.sayHi)
```

Наследование через прототипы:

```
function Animal(name) {
  this.name = name;
}

Animal.prototype.eat = function() {
  console.log(this.name + " eats");
};

function Dog(name, breed) {
  Animal.call(this, name); // вызываем родителя
  this.breed = breed;
}

// наследуем prototype
Dog.prototype = Object.create(Animal.prototype);

// важно вернуть constructor (осле перезаписи prototype он теряется)
Dog.prototype.constructor = Dog;
```

```
// Добавляем метод уже в Dog.prototype
Dog.prototype.bark = function() {
  console.log("Woof!");
};

// Создаём экземпляр
const dog = new Dog("Rex", "Labrador");

dog.eat();  // dog → нет, Dog.prototype → нет, Animal.prototype → есть
dog.bark(); // dog → нет, Dog.prototype → есть

// Полная цепочка:
// dog
//   → Dog.prototype
//       → Animal.prototype
//           → Object.prototype
//               → null
```

Наглядный поиск свойства по цепочке:

```
function A() {} // конструктор "родителя"
function B() {} // конструктор "наследника"

A.prototype.say = function() {
  console.log("from A"); // метод в прототипе A
};

B.prototype = Object.create(A.prototype); // наследуем prototype от A

B.prototype.constructor = B; // восстанавливаем правильный constructor

const obj = new B(); // создаём объект с прототипом B.prototype

obj.say(); // метод найден в A.prototype через цепочку:
// obj → нет
// B.prototype → нет
// A.prototype → есть
```

Переопределение метода:

```
function Animal() {}

Animal.prototype.say = function() {
  console.log("animal");
};

function Dog() {}

Dog.prototype = Object.create(Animal.prototype);

Dog.prototype.say = function() {
  console.log("dog");
};

const d = new Dog();

d.say(); // dog
// JS берёт первое найденное свойство в цепочке
```

Домашнее задание:

https://ru.hexlet.io/programs/js-oop

Материалы лекций:

https://github.com/ShViktor72/Education2025

Обратная связь:

colledge20education23@gmail.com